Notes on Bayesian Optimization

Luis Mendez

Overview

Manual, grid, and random search are all *uninformed*: the next combination they evaluate does not depend on the scores already observed. Every trial is drawn independently, so information gathered at trial t is thrown away when choosing trial $t+1$. Bayesian optimization is the first strategy in these notes that is *informed* — it fits a probabilistic model to the trials seen so far and uses that model to decide where to look next.

This matters because in hyperparameter tuning the objective is expensive. Each evaluation means fitting a model k times for k-fold cross-validation, so a budget of 50 evaluations may be all we can afford. When evaluations are scarce, it pays to spend computation *deciding where to sample* rather than sampling blindly.

The Objective and Why It Is Hard

Let $f(\mathbf{x})$ be the cross-validated score of a model configured with hyperparameters $\mathbf{x} \in \mathcal{X}$. We want

$$\mathbf{x}^* = \arg\min_{\mathbf{x}\in\mathcal{X}} f(\mathbf{x}). \tag{1}$$

This objective has properties that rule out classical optimization:

- **Black box**: we have no analytic form for f, only the ability to evaluate it at a point.
- **No gradients**: ∇f is unavailable, so gradient descent and its relatives do not apply. Some hyperparameters are not even continuous (`max_depth` is an integer, `loss` is categorical), so a gradient would not exist regardless.
- **Expensive**: one evaluation costs k model fits, sometimes minutes or hours.
- **Noisy**: f is estimated from a finite sample, so re-evaluating the same $\mathbf{x}$ with a different CV split gives a slightly different value.
- **Possibly non-convex** with multiple local optima.

Bayesian optimization addresses precisely this setting: derivative-free minimization of an expensive, noisy black box under a small evaluation budget.

Sequential Model-Based Optimization (SMBO)

Bayesian optimization is an instance of the SMBO pattern. Two ingredients replace direct optimization of f:

- A **surrogate model** (or response surface) — a cheap probabilistic approximation of f fitted to the trials observed so far, which returns not just a prediction but an *uncertainty* at every point.
- An **acquisition function** — a cheap utility computed from the surrogate that scores how promising each candidate point is. We minimize f indirectly by maximizing the acquisition function, which is cheap to evaluate and differentiable, so it can be optimized with ordinary methods.

The loop, given a budget of n calls:

1) **Initialize.** Evaluate f at n_0 points chosen without a model (random, or a space-filling design). A surrogate cannot be fitted to zero observations, so this bootstrapping step is required. In `scikit-optimize` this is `n_initial_points`, default 10.
2) **Fit the surrogate** to all observations $\mathcal{D}_t = \{(\mathbf{x}_i, y_i)\}_{i=1}^{t}$.
3) **Maximize the acquisition function** over $\mathcal{X}$ to pick $\mathbf{x}_{t+1}$.
4) **Evaluate** $y_{t+1} = f(\mathbf{x}_{t+1})$ — the one expensive step.
5) **Augment** $\mathcal{D}_{t+1} = \mathcal{D}_t \cup \{(\mathbf{x}_{t+1}, y_{t+1})\}$ and repeat until the budget is exhausted.

The name is Bayesian because step 2 is a posterior update: the surrogate encodes a prior over functions, and each observation refines it into a posterior belief about where the optimum lies.

The Surrogate: Gaussian Processes

The classic surrogate is a Gaussian process (GP). A GP places a prior over *functions*, specified by a mean function and a covariance (kernel) function $k(\mathbf{x}, \mathbf{x}')$:

$$f(\mathbf{x}) \sim \mathcal{GP}(m(\mathbf{x}),\, k(\mathbf{x}, \mathbf{x}'))\,. \tag{2}$$

Its defining property is that any finite collection of function values is jointly Gaussian. Conditioning on observed data therefore yields a closed-form Gaussian posterior at any new point $\mathbf{x}$, with mean $\mu(\mathbf{x})$ and standard deviation $\sigma(\mathbf{x})$.

Those two quantities are exactly what the acquisition function needs:

- $\mu(\mathbf{x})$ — the best guess of the score at $\mathbf{x}$ (drives *exploitation*).
- $\sigma(\mathbf{x})$ — how uncertain that guess is (drives *exploration*).

$\sigma(\mathbf{x})$ shrinks near observed points and grows in unexplored regions, which is what lets the method reason about where it has *not* yet looked. This is the crucial advantage over a non-probabilistic regressor: a plain interpolator would predict a value everywhere but could not say where it is ignorant.

Kernels

The kernel encodes the assumed smoothness of f and is the main modeling choice. Two common options:

- **RBF (squared exponential)**, $k(\mathbf{x}, \mathbf{x}') = \exp\!\big(-\|\mathbf{x} - \mathbf{x}'\|^2 / 2\ell^2\big)$, which assumes f is infinitely differentiable — very smooth.

- **Matérn**, a family parameterized by ν controlling smoothness. Finite ν (typically $\nu = 3/2$ or $5/2$) yields sample paths that are continuous but only finitely differentiable, and $\nu \to \infty$ recovers the RBF. This is `gp_minimize`'s default because the strong smoothness assumption of the RBF is usually unrealistic for hyperparameter response surfaces.

The **length scale** ℓ sets how far correlation extends: small ℓ means the function may wiggle rapidly, so observations constrain only their immediate neighborhood; large ℓ means a few observations pin down a wide region. Kernel hyperparameters are themselves fitted (by marginal-likelihood maximization) when the GP is refitted, so there is a nested optimization inside the outer search — an accepted cost, since the GP fit is trivial compared to a model fit.

Substituting the kernel changes the search's behavior: an overly smooth kernel can cause the surrogate to miss sharp optima, while a rough kernel keeps $\sigma(\mathbf{x})$ large and makes the search behave more like random search.

Acquisition Functions

The acquisition function converts $(\mu(\mathbf{x}), \sigma(\mathbf{x}))$ into a single score to maximize. Writing $f_{\min}$ for the best value observed so far (we are minimizing):

Expected Improvement (EI)

The default, and usually the best first choice:

$$\mathrm{EI}(\mathbf{x}) = \mathbb{E}[\max(f_{\min} - f(\mathbf{x}), 0)]\,. \tag{3}$$

It measures the expected size of the improvement, not merely its probability. Under the GP posterior it has a closed form. With $Z = (f_{\min} - \mu(\mathbf{x}))/\sigma(\mathbf{x})$,

$$\mathrm{EI}(\mathbf{x}) = (f_{\min} - \mu(\mathbf{x}))\,\Phi(Z) + \sigma(\mathbf{x})\,\phi(Z), \tag{4}$$

where Φ and ϕ are the standard normal CDF and PDF. The two terms show the tradeoff explicitly: the first rewards points predicted to be better than the incumbent (exploitation), the second rewards points with high uncertainty (exploration). A slack parameter ξ can be added to $f_{\min}$ to bias toward exploration.

Probability of Improvement (PI)

$$\mathrm{PI}(\mathbf{x}) = \Phi\left(\frac{f_{\min} - \mu(\mathbf{x})}{\sigma(\mathbf{x})}\right). \tag{5}$$

This asks only *whether* a point improves, ignoring by how much. It therefore tends to favor small, safe gains near the incumbent and is more prone to getting stuck than EI.

Lower Confidence Bound (LCB)

$$\mathrm{LCB}(\mathbf{x}) = \mu(\mathbf{x}) - \kappa\,\sigma(\mathbf{x}), \tag{6}$$

minimized over $\mathbf{x}$ (the maximization analogue is UCB). Here the exploration/exploitation balance is a single explicit knob: $\kappa = 0$ is pure exploitation (go to the predicted minimum), and large κ is pure exploration (go where uncertainty is greatest). This makes LCB attractive when the tradeoff needs to be controlled directly.

Exploration vs. exploitation

This is the central tension of any informed search. Pure exploitation converges quickly to a local optimum and can miss a better basin entirely; pure exploration degenerates into random search and wastes the surrogate. Because $\sigma(\mathbf{x})$ shrinks wherever the search has already sampled, all three acquisition functions self-correct to some degree: exploited regions become less attractive as their uncertainty collapses, which naturally pushes the search elsewhere.

Implementation in Scikit-Optimize

`scikit-optimize` (`skopt`) provides `gp_minimize`, which runs GP-based Bayesian optimization with a Matérn kernel by default.

Defining the search space

The space is a *list* of dimension objects (not a dict, unlike `GridSearchCV`), because ordering matters — results are returned as a positional list:

- `Integer(low, high, name=...)` for integer hyperparameters.
- `Real(low, high, name=...)` for continuous ones; a `prior='log-uniform'` is available for parameters spanning orders of magnitude, such as a learning rate or regularization strength.
- `Categorical([...], name=...)` for unordered choices.

Note that these are *ranges*, not enumerated candidate values. This is a genuine expressiveness gain over grid search: resolution is not capped by a grid chosen up front.

The objective function

The objective takes a hyperparameter configuration, cross-validates the model, and returns a scalar to **minimize**. Since we normally want to *maximize* a score such as accuracy or ROC-AUC, the function must return its **negative**. Forgetting this sign flip is the most common mistake — the search will run without error and cheerfully return the *worst* configuration it can find.

The `@use_named_args(param_grid)` decorator adapts an objective written with keyword arguments to the positional list `skopt` passes internally, letting the body forward them straight to `set_params(**params)`.

Key arguments

- `n_calls` — total evaluations of f, i.e. the whole budget.
- `n_initial_points` — how many of those are the initial model-free samples. The informed phase gets `n_calls` − `n_initial_points` evaluations, so setting these too close together leaves the surrogate almost no chance to act.
- `acq_func` — `'EI'`, `'PI'`, or `'LCB'`.

- `random_state` — needed for reproducibility, since the initial design is random.

The returned result object exposes `.x` (best hyperparameters), `.fun` (best objective value, still negated if the objective was negated), and `.func_vals` (all observed values), with `plot_convergence` charting the running minimum against iteration.

Worked Example: Tuning a GBM

Tuning a `GradientBoostingClassifier` on the breast cancer dataset with `gp_minimize` makes the mechanics concrete. The search space mixes all three dimension types:

Hyperparameter	Type	Range
`n_estimators`	Integer	$[10, 120]$
`min_samples_split`	Real	$[0.0001, 0.999]$
`max_depth`	Integer	$[1, 5]$
`loss`	Categorical	log_loss, exponential

The objective returns the negative mean 3-fold CV accuracy. Running with `n_initial_points=10`, `acq_func='EI'`, and `n_calls=50` — so 10 random probes followed by 40 model-guided ones — yields a best objective of -0.9698, i.e. mean CV accuracy ≈ 0.970, at ($n\_estimators = 120,\ min\_samples\_split = 0.786,\ max\_depth = 5,\ loss =$ log_loss).

Two observations worth noting. First, two of the optimal values sit at the *upper boundary* of their ranges ($n\_estimators = 120$, $max\_depth = 5$), which is a signal that the ranges were set too narrow and the true optimum may lie outside the box — the natural next step is to widen them and re-run. Second, the selected $min\_samples\_split \approx 0.79$ is high, meaning splits are only attempted on large nodes, which regularizes the trees; a continuous `Real` dimension can express this value, whereas a coarse grid over $\{0.1, 0.3, 0.5\}$ could never have found it. That is the resolution advantage of range-based spaces in practice.

Pros / Cons

- **Pros**: far more sample-efficient than grid or random search, typically reaching a comparable or better optimum in many fewer evaluations, which is decisive when a single evaluation is expensive; handles mixed integer/real/categorical spaces without gradients; searches continuous ranges rather than a fixed grid; the surrogate is a reusable model of the response surface, indicating which hyperparameters matter.
- **Cons**: **inherently sequential** — each trial depends on all previous ones, so it does not parallelize the way grid and random search trivially do (batch variants exist but complicate the method); per-iteration overhead of refitting the GP and optimizing the acquisition function, which is only worthwhile when f is expensive; GP cost scales roughly cubically in the number of observations, so it suits budgets of tens-to-hundreds of trials rather than many thousands; performance depends on modeling choices (kernel, acquisition function, n_0) that are themselves hyperparameters; and GPs handle high-dimensional and heavily conditional spaces poorly, which motivates the alternative surrogates of the next section.

Comparison With the Uninformed Methods

- Grid and random search choose all their points independently of observed scores; Bayesian optimization chooses each point conditioned on every result so far. Uninformed search buys parallelism at the cost of sample efficiency, informed search does the reverse.
- With a very large budget and abundant parallel compute, random search is often the pragmatic choice — it is simpler, has no surrogate overhead, and scales across machines. With a small budget and expensive fits, Bayesian optimization is usually the better use of the same wall-clock time.
- A practical hybrid: use random search first to establish sensible ranges, then hand those ranges to Bayesian optimization for refinement — the same coarse-to-fine idea as two-stage grid search, but with an informed second stage.

Practical Notes

- **Sign convention**: `skopt` always minimizes, so any score to be maximized must be negated in the objective and unnegated when reporting.
- **Boundary hits**: an optimum at the edge of a range usually means the range, not the optimum, was the constraint. Widen and re-run.
- **Version constraint**: `scikit-optimize` 0.10.2 is its final release and predates the `__sklearn_tags__` estimator-tag protocol introduced in scikit-learn 1.6. On scikit-learn $\geq$ 1.6, `dummy_minimize` and `gbrt_minimize` raise `AttributeError`, so scikit-learn must be held below 1.6 to run these notebooks (see `requirements.txt`).
- **Deprecated loss name**: the GBM loss formerly called `'deviance'` is now `'log_loss'` in current scikit-learn; older course material using the old string will fail.

Looking Ahead

The GP is one surrogate among several, and its weaknesses in high-dimensional, conditional, and mixed spaces motivate the alternatives that follow: random forest and gradient-boosted tree surrogates (SMAC), and Tree-structured Parzen Estimators (TPE), which model $p(\mathbf{x} \mid y)$ rather than $p(y \mid \mathbf{x})$. All of them keep the SMBO loop above and swap only the surrogate and the way the acquisition step is computed.